

Day04、子查询、DDL

- ✓ 必须掌握子查询
 - ✓ from子查询、where子查询、字段子查询
 - ✓ exists 子查询（难点）
- ✓ 掌握集合操作
- ✓ DDL

一、子查询

子查询指的就是在一个查询之中嵌套了其他的一个查询。我们回顾下查询语句

```
1 select 列1, 列2...
2 from 表名
3 where 条件
```

根据以上语法，子查询的位置可以在 FROM、WHERE 子句中，还可以在select子句中，形如：

```
1 select 列1, 列2...
2 from 表名
3 where 条件
```

这样，子查询分为3类：

- from子查询：from后接子查询
- where子查询：where后接子查询
- 字段子查询：select 后接子查询

注意：子查询要使用圆括号括起来；

1、from 接子查询

结论：子查询作为一个表使用。

```

1  -- 案例：查询薪资大于等于3000的雇员名称、部门编号、薪资
2  -- 1 查询薪资大于等于3000的雇员
3  select * from emp e where e.sal >= 3000;
4  -- 2 以上结果集作为临时表，继续查雇员名称、部门编号、薪资
5  select t.ename, t.sal, t.deptno
6  from (select * from emp e where e.sal >= 3000) t

```

需求:每个部门平均薪水的等级

```

1  -- 1 求部门平均薪资
2  select e.deptno, avg(e.sal)
3  from emp e
4  group by e.deptno;
5
6  -- 2 求平均薪水的等级
7  select *
8  from (select e.deptno, avg(e.sal) "AVGSAL" from emp e group by
9        e.deptno) vt
10     join salgrade sg on vt.avgsal between sg.losal and
11        sg.hisal

```

2、where 接子查询

在from子查询中，子查询结果是多行多列，作为表使用，

子查询结果还可能是一行1列，或多行1列

- 子查询的结果是一行1列，当做一个值使用
- 子查询的结果是多行1列，当做列表使用

2.1 单行子查询

需求：查询薪水是在整个雇员的平均薪水之上的雇员

```

1  -- 1 整个雇员的平均薪水
2  select avg(e.sal) from emp e
3  -- 2 查询满足条件雇员
4  select * from emp e where e.sal > (select avg(e.sal) from emp
5        e)

```

结论：如果子查询结果是1行1列, 当做一个值, 放在where中结合关系运算作为条件使用

需求2：查询销售部 SALES 所有的雇员

```

1 select e.ename,e.job,e.sal
2 from emp e
3 where e.deptno=(select d.deptno from dept d where d.dname =
  'SALES');

```

小练习：找出20部门所有员工中收入最高的职员

```

1 -- 1 部门20所有雇员的最高薪资
2 select max(e.sal) from emp e where e.deptno = 20
3 -- 2 部门20最高薪资的雇员
4 select * from emp e
5 where e.deptno = 20 and e.sal >= (select max(e.sal) from emp e
  where e.deptno = 20)

```

小练习2: 查询销售部SALES薪资最高的职员（重要）

```

1 -- 1 查询销售部SALES的部门号 => 30
2 select deptno from dept where dname = 'SALES'
3
4 -- 2 统计30部门最高薪资
5 select max(sal) from emp where deptno = (select deptno from
  dept where dname = 'SALES');
6
7 -- 3 从emp表查30部门、薪资是2850的雇员
8 select *
9 from emp
10 where deptno = (select deptno from dept where dname = 'SALES')
  and
11      sal = (select max(sal) from emp where deptno = (select
    deptno from dept where dname = 'SALES'))

```

2.2 多行子查询

多行子查询：子查询的结果集是多行1列，可以作为列表 list 使用，配合 **in (list)**、**exists(list)**使用

```

1  多行1列称为列表，形如：
2  /*
3  -----
4  | deptno |
5  -----
6  |    10  |
7  -----
8  |    30  |
9  -----
10 */

```

需求1：查询销售部(SALES)和 财务部(ACCOUNTING) 的雇员信息

```

1  -- 1 查询两个部门的部门号 10 30
2  select deptno from dept where dname = 'SALES' or dname =
   'ACCOUNTING';
3  -- 2 查询雇员在部门10、30的雇员
4  select *
5  from emp
6  where deptno in (select deptno from dept where dname = 'SALES'
   or dname = 'ACCOUNTING');

```

需求2：查询雇员中所有的管理者信息

```

1  -- 1 查询所有管理者(list)
2  select distinct e.mgr from emp e
3  -- 2 查找在list中的雇员信息
4  select * from emp e where e.empno in (select distinct e.mgr
   from emp e )

```

小结：观察发现，from子查询、where子查询

- 子查询先执行，再执行主查询

3、字段子查询

当子查询出现在字段（即select列表中）时，它通常用于查询某个特定的值。

```

1  位置：select (select查询)，列2...

```

特点：先执行主查询，后执行子查询

案例：显示每个员工的姓名及其所在部门的名称

```

1  -- 显示每个员工的姓名及其所在部门的名称
2  SELECT
3      e.ENAME,
4      (SELECT d.DNAME FROM dept d WHERE e.DEPTNO = d.DEPTNO) AS
        DepartmentName
5  FROM emp e;
6
7  -- 显示每个员工的姓名及其薪资等级
8  SELECT
9      e.ENAME,
10     (SELECT s.GRADE FROM salgrade s WHERE e.SAL BETWEEN
        s.LOSAL AND s.HISAL) AS SalaryGrade
11  FROM emp e;

```

小练习

实战1: 显示每个员工的姓名、职位、部门名称、该部门的最高薪资

```

1
2  select
3      e.ename,
4      e.sal,
5      e.job,
6      e.deptno,
7      (select dname from dept where deptno = e.deptno) as dname,
8      (select max(sal) from emp where deptno = e.deptno) as maxsal
9  from emp e;

```

实战2: 显示每个员工的姓名、职位、部门名称以及部门内该职位的最高薪资（重要）

```

1  select
2      e.ename,
3      e.job,
4      e.deptno,
5      (select d.dname from dept d where d.deptno = e.deptno),
6      (select max(a.sal) from emp a where a.deptno = e.deptno and
        a.job = e.job)
7  from emp e;

```

4、exists 接子查询

exists 关键字表示存在的意思。语法：

```
1 exists 子查询
2 # 含义：子查询是否存在结果，存在则返回true，否则false
```

理解: exists关键字在你想要检查**不同表中相关记录的存在性**，而不实际检索那个表中的全部数据时非常有用。

好处：它可以作为优化查询的强大工具，**因为可以在不返回子查询结果的情况下检查是否存在满足特定条件的行**

需求1：查询存在雇员的部门，显示部门ID、部门名称、部门位置

```
1 select t1.deptno
2 from dept t1
3 where exists (select 1 from emp t2 where t1.deptno = t2.deptno)
```

需求2：找出所有在“销售SALES”部门工作的雇员。

```
1 SELECT Empno, Ename
2 FROM emp
3 WHERE EXISTS (SELECT 1 FROM dept WHERE dept.Deptno = emp.Deptno
AND dname = '销售');
```

小练习：使用 exists 子查询

- 找出所有没有津贴的雇员。
- 找出所有有经理的雇员。也就是说，雇员的Mgr字段不为空。

```
1 -- 找出所有没有津贴的雇员。
2 select e.*
3 from emp e
4 where exists (select 1 from emp a where a.empno = e.empno and
a.comm is null)
5
6 -- 找出所有 有管理者的雇员。也就是说，雇员的Mgr字段不为空。
7 select e.*
8 from emp e
9 where exists (select 1 from emp a where a.empno = e.empno and
a.mgr is not null)
```

二、集合操作

```
1 -- 集合操作：交集 并集 差集
2 -- 注意：集合理解为结果集
3
```

```

4  -- (1) 交集
5  语法: A intersect B
6  -- 演示: 交集
7  select * from emp where deptno = 10
8  intersect
9  select * from emp where deptno = 10 or deptno = 20;
10
11 -- (2) 并集
12 语法:
13  A union B          取并集, 并去掉重复记录
14  A union all B      取并集, 并保留重复记录
15
16 面试题: union 和 union all 区别?
17
18 -- 演示: 并集
19 select * from emp where deptno = 10 or deptno = 20
20 union all
21 select * from emp where deptno = 20 or deptno = 30;
22
23 -- 并集
24 -- 要求: 记录的列数、数据类型要一致
25 select ename, sal from emp where deptno = 10
26 union
27 select ename, comm from emp where deptno = 20;
28
29 -- 差集: A - B
30 语法: A minus B
31
32 -- 演示:
33 select * from emp where deptno = 10 or deptno = 20
34 minus
35 select * from emp where deptno = 10;

```

三、DDL

DDL : data defintion language, 专门用于创建 (或定义) 数据库对象 (例如: 表)

1、列的数据类型

列的数据类型常见的有三种:

- | | | |
|---|------------------|---------------------------------------|
| 1 | char(max_length) | 可存储长度固定的字符串, max_length指定长度, 最大2000字节 |
|---|------------------|---------------------------------------|

2		例如：存手机号
3	<code>phone char(11)</code>	长度
4	<code>18620011234</code>	占11字节
5	<code>10086</code>	占11字节(空格填充)
6		
7	<code>varchar2(length)</code>	可存储长度不固定的字符串， <code>length</code> 指定长度，最大4000字节
8		例如：存用户名 <code>ikun</code>
9	<code>name varchar2(60)</code>	长度
10	<code>ikun</code>	4字节
11		
12	<code>number(p, s)</code>	存数值类型(存整数、小数)
13		<code>p</code> : 数值的位数，可省， <code>p</code> 默认是38
14		<code>s</code> : 小数的位数，可省，默认值0位
15		例如： <code>number(5, 2)</code> 整数3位，小数2位 =>
	<code>[000.00, 999.99]</code>	
16		
17	<code>date</code>	存日期和时间，只能精确到秒

其他的类型（了解）：

数据类型	作用
<code>timestamp</code>	时间戳，存储日期和时间，精确到微秒
<code>long</code>	存长文本（字符），最长2GB

(1) 字节和字符的关系？

1	字符	编码(字符集)	字节
2	1个字母或数字	任意编码	1个
3	1个汉字	GBK	2个
4		UTF8	3个

案例：字节和字符关系

1	<code>-- length()</code>	计算字符个数
2	<code>-- lengthb()</code>	计算字符的字节大小
3	<code>--</code>	查看当前Oracle系统字符集
4	<code>select userenv('language') from dual;</code>	
5		
6	<code>select length('a'), lengthb('a') from dual;</code>	
7	<code>select length('你'), lengthb('你') from dual;</code>	

(2) 各种数据类型的表达形式:

- 字符型用单引号加字符表示, 例如: 'ABC'
- 数字型直接用阿拉伯数字表示, 例如: 123、3.14
- **日期型不能直接写, 必须写成字符串, 然后使用to_date转换**

例如:

```
1 select to_date('1987-01-01', 'YYYY-MM-DD') from dual;  
2 select to_date('20220804140001', 'YYYYMMDDHH24MISS') as  
   test_time from dual;
```

(3) 各种数据类型的不同特点:

- 字符型可以进行拼接 ||
- 数字型可以进行算术运算 + - * /
- **日期型也能进行算术运算**, 但是只能日期加减日期, 或者**日期加减数字**

```
1 -- 日期可以做加减法, 但是不能做乘除  
2 -- 注意日期加减的单位是天  
3 select sysdate, sysdate + 1 from dual;
```

(4) 各种数据类型比较大小方式:

- 字符型比较逐位字符的ascii码的大小, 规则: 按字母表逐位
- 数字型比较数值的大小, 规则: 直接比较
- 日期型也是直接比较大小, 规则: 越早的时间越小;

小练习: 问AI

```
1 -- 字符比较大小  
2 select 1 from dual where 'ad' > 'abc';  
3  
4 -- -- 查询出emp表中所有hiredate 大于 1987年1月1日的员工信息  
5 select *  
6 from emp  
7 where hiredate > to_date('1987-01-01', 'YYYY-MM-DD');
```

DDL (data definition language) 是数据定义语言。包括create(创建命令),alter(修改命令),drop(删除命令)等。

2、创建表、删除表、截断表

创建表的基本语法

```
1 create table table (  
2     列名1 类型,  
3     列名2 类型,  
4     ...  
5 );
```

需求：创建一个学生信息表

```
1  -- 创建一个学生信息表（学号,姓名,性别,年龄,生日,手机号,家庭住址）  
2  create table t_stu_info(  
3      sid number(4),  
4      sname varchar2(60),  
5      gender number(1),  
6      age number(3),  
7      birthday date,  
8      phone char(11),  
9      address varchar2(300)  
10 );
```

有时候，为了复制数据或表结构，根据查询结果集创建

```
1  -- 复制表语法：  
2  create table 表名  
3  as  
4  select * from 源表 where 条件  
5  
6  -- 需求2：复制emp为t_emp1，要求复制表结构和数据  
7  create table t_emp1  
8  as  
9  select * from emp;  
10  
11 -- 需求3：复制emp为t_emp2，要求只复制表结构  
12 create table t_emp2  
13 as  
14 select * from emp where 1=2;
```

删除表

```
1  -- 删除表和截断表
2  删除表语法：
3  drop table 表名;      -- 删除所有数据并删除表结构
4
5  -- 需求：删除t_student表
6  drop table t_student;
7
8  -- 需求2：删除t_emp1表
9  drop table t_emp1;
```

截断表，语法

```
1  截断表语法：
2  truncate table 表名; -- 清空表(删除所有数据)，结构还在
3
4  -- 需求3：清空t_emp1表
5  truncate table t_emp1;
```

3、修改表（了解）

```
1  语法：
2  alter table 表名 add 列名 类型;      -- 添加列
3  alter table 表名 drop column 列名;    -- 删除列
4  alter table 表名 modify 列名 新类型  -- 修改列的数据类型
5  alter table 表名 rename column 列名 to 新列名 -- 修改列名
6  alter table 表名 rename to 新表名    -- 修改表名
7
8  -- 演示：
9  alter table t_student add address varchar2(100);
10 alter table t_student drop column age;
11 alter table t_student modify address varchar2(200);
12 alter table t_student rename column address to addr;
13 alter table t_student rename to t_stu;
```

建议：使用可视化操作

